CSC_51053_EP, Exercise Session: Indexing, Query Evaluation

Ioana Manolescu, Madhulika Mohanty, Garima Gaur

October 24, 2025

1 SQL queries: buying a gift together

We consider a database about users who pool money to buy someone a gift; the database also has information about possible gifts, and about what kind of gift each user thinks is best. *The database described below describes only one effort to buy such a gift.*

The tables are created through the following commands:

create table **guser**(uid int primary key, uname varchar);
– *people who pool money to buy a gift*
create table **category**(cid int primary key, cname varchar);
– *gift categories, e.g., 'sport', 'art', 'cooking' etc.*
create table **gift**(id int primary key, descr varchar, price int, cID int references category(cid));
– *possible gifts that could be bought*
create table **contribution**(uid int primary key, amount int not null);
– *each user may contribute (or not) an amount of money to the gift*
create table **preference**(uid int references guser(uid), cid int references category(cid), porder int not null check (1<=porder and porder<=3), primary key (uid, cid, porder));
– *each user may specify at most 3 preferences for the category of the gift to be bought. A user preference order (porder) of 1 is the option the user favors most; 2 is the "next" preference order, and 3 is the least favored preference order*

If a user u put a category c first in her preference order (porder=1), we view this as a *support score* of 3; similarly, porder=2 means a support score of 2, while porder=3 means a support of 1.

1. Write the SQL query Q1: for each category whose name starts with 'M', with non-zero total support across all users, find the category name and the total category support. Order the categories from the highest to the lowest total support.

   Where exactly in the query is the non-zero total support condition expressed?

2. Write the variant query Q2: for each category (including those with no support), find the category name and the total support. Provide one variant using an outerjoin, and another without.

3. A gift is *affordable* if its price is less than the sum of all contributions. Write the query Q3: For each affordable gift, return the gift id, description and price.

4. A gift makes best use of the money if the gift (i) is affordable, and (ii) has a price that is the closest to the sum of all contributions, among all the affordable gifts.

   (a) Write the query Q4: Find the id(s), description(s) and price of gift(s) making best use of the money.

   Can we write the same query without a WITH? What would be the impact on the physical plan?

5. While searching for "the ideal gift to buy", we see there are two distinct ways to order them:

   - from the most expensive one (still within the budget), to the least expensive one; we prefer gifts that spend as much of the budget as possible;
   - according to their categories, from the category with most support to the category with least support.

In general, there may be no single best gift, as one who is ranked at the top on one dimension may be ranked lower in the other dimension.

We say a gift g_1 is *Pareto-optimal* if there exists no other gift g_2 that is better than g_1 on both dimensions.

(a) Write the query Q5 that returns the IDs and descriptions of all the Pareto-optimal gifts. Propose a physical plan for it.

Propose one or two more variants of this query.

6. *(Modeling)* Modify the schema described above so that it can capture *several* gift pools (e.g., in the style of the Leetchi platform). Describe briefly how the above queries Q1-Q5 could be re-stated in natural language (think of at least two variants).

2 B+ trees insertions and deletions

1. Consider a B+-tree whose maximum number of keys n in a node is 4. The tree is initially empty. Show the tree obtained after inserting *successively, one by one*, the keys $1^*, 2^*, \ldots, 13^*$.

2. Now show the tree resulting from inserting these keys through bulk-loading.

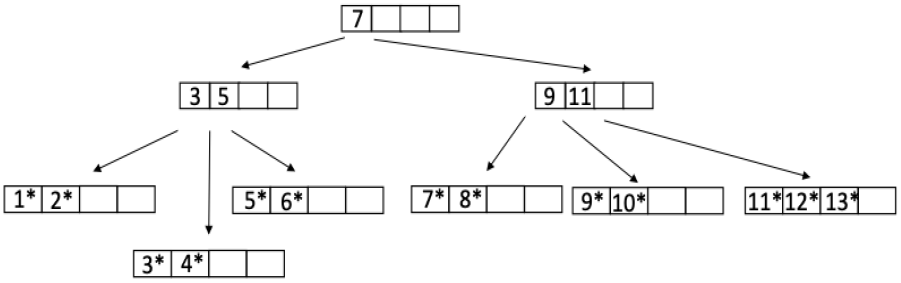


Figure 1: Tree for B+ tree deletions.

3. Delete the value 10^* in the B+ tree represented in Figure 1. Delete the value 3^* in the B+ tree represented in Figure 1. Delete 13^* from the B+ tree represented in Figure 1.

4. Delete the values 13^*, 10^* and 3^* *one after another* from the tree in Figure 1 (thus each deletion applies on the result of the previous one).

3 Selection cost [1]

A table Executives has attributes *ename* (which is a key), *title*, *dname* and *address*. All are string attributes have the same length. The relation contains **10000** pages. There are **10** buffer pages.

1. Consider the following query:

```
SELECT E.title, E.ename
FROM Executives E
WHERE E.title = 'CFO'
```

Assume that: (*i*) **10%** of Executives tuples meet the selection condition; (*ii*) a record ID has the same size as an attribute. In each of the following cases, assuming the described index is *dense*, that it is *the only one available*, that the alternative (2) seen in the course is used, and that *its pages are 100% full*, describe and evaluate the cost of the best plan.

Scenario A a clustered B+ tree index on *title*;

Scenario B an unclustered B+ tree index on *title*;

Scenario C a clustered B+ tree index on *ename*;

Scenario D a clustered B+ tree index on *address*;

Scenario E a clustered B+ tree index on *ename,title*;

2. Do the same for the following query and indexes:

```
SELECT E.ename
FROM Executives E
WHERE E.title = 'CFO' and E.dname='Toy'
```

Assume that **10%** of Executives tuples meet the condition `E.title = 'CFO'`, **10%** meet the condition `E.dname = 'Toy'`, and that **5%** meet both conditions.

Scenario G a clustered B+ tree index on *title*;

Scenario H a clustered B+ tree index on *dname*;

Scenario I a clustered B+ tree index on *(title,dname)*;

Scenario J a clustered B+ tree index on *(title,ename)*;

Scenario K a clustered B+ tree index on *(dname,title,ename)*;

Scenario L a clustered B+ tree index on *(ename,title,dname)*.

We assume that traversing the index costs 2 I/Os.

References

[1] Raghu Ramakrishnan and Johannes Gehrke. *Database management systems (3rd ed.)*. McGraw-Hill, 2003.